

Hardware Implementation of Four Degree of Freedom Deep Reinforcement Learning Arm Control Model Trained Through Human Demonstrations

Roel Hernandez
Apr 22, 2022

MS Project Presentation
Intelligent Systems Lab
Department of Engineering Science
School of Science and Technology
Sonoma State University

Motivation

- First class about Intelligent Artificial (1993)
- Robotic class (Spring 2021)
- Autonomous Robots in Assistive Technology, Precision Manufacturing, Surgery, and Extreme Environments

Related Work

- (Christiano et al 2017) a non-expert human is used for feedback by defining goals; where they compare and select amongst possible trajectories.
- (Duran et al 2021) explore the capability and efficacy of controlling a four degree of freedom (4-DOF) robot arm simulation with a deep reinforcement learning agent.
- (Zhang et al 2015) introduces a machine learning based system for controlling a robotic manipulator with visual perception only.
- (Gu et al 2016) demonstrate that a recent deep reinforcement learning algorithm based on offpolicy training of deep Q-functions can scale to complex 3D manipulation tasks

Introduction

- 4 DOF Robot Arm controlled by an intelligent agent
 - Read and Write 4 joint variables
 - calculate tip position via forward kinematics
 - select 1 of 81 possible moves
- Test Deep Reinforcement Learned Model
 - Run test simulation using trained model
 - Run trained model on robotic arm

Research Objectives

1. Implement the trained model to perform task with a 4 DOF robot arm, given source and destination coordinates.
2. To use portable hardware (Raspberry Pi) to implement the trained model.

Theoretical Background

Theory 1 - Reinforcement Learning

Basic reinforcement is modeled as a Markov decision process (MDP):

- A set of environment and agent states, S .
- A set of actions, A , of the agent.
- $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$ is the probability of transition (at time t) from state s to state s' under action a .
- $R_a(s, s')$ is the immediate reward after transition from s to s' with action a .

Theoretical Background

Theory 2 – Robot configuration

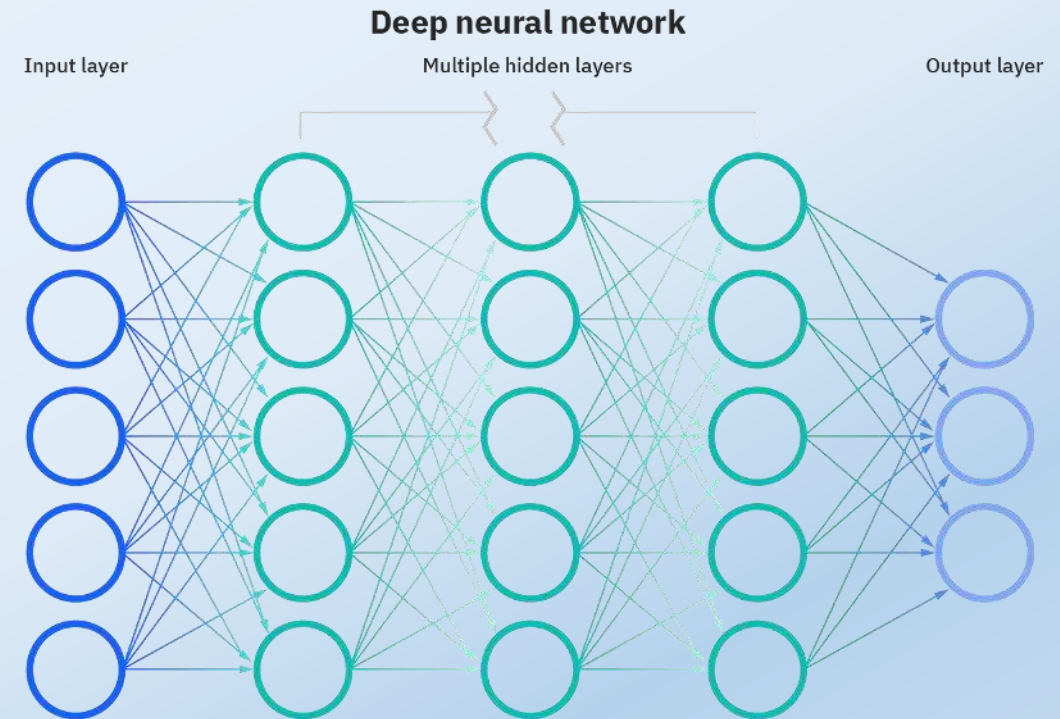
- The Denavit-Hartenberg (DH) convention defines a configuration for our robot arm.
- We will use the raw joint variables ($\theta_1, \theta_2, \theta_3, \theta_4$) and forward kinematics to locate the end effector(x_t, y_t, z_t)

Axes	θ	α	a	d
1	θ_1	-90	0	106
2	θ_2	180	146	0
3	θ_3	0	146	0
4	θ_4	0	80	0

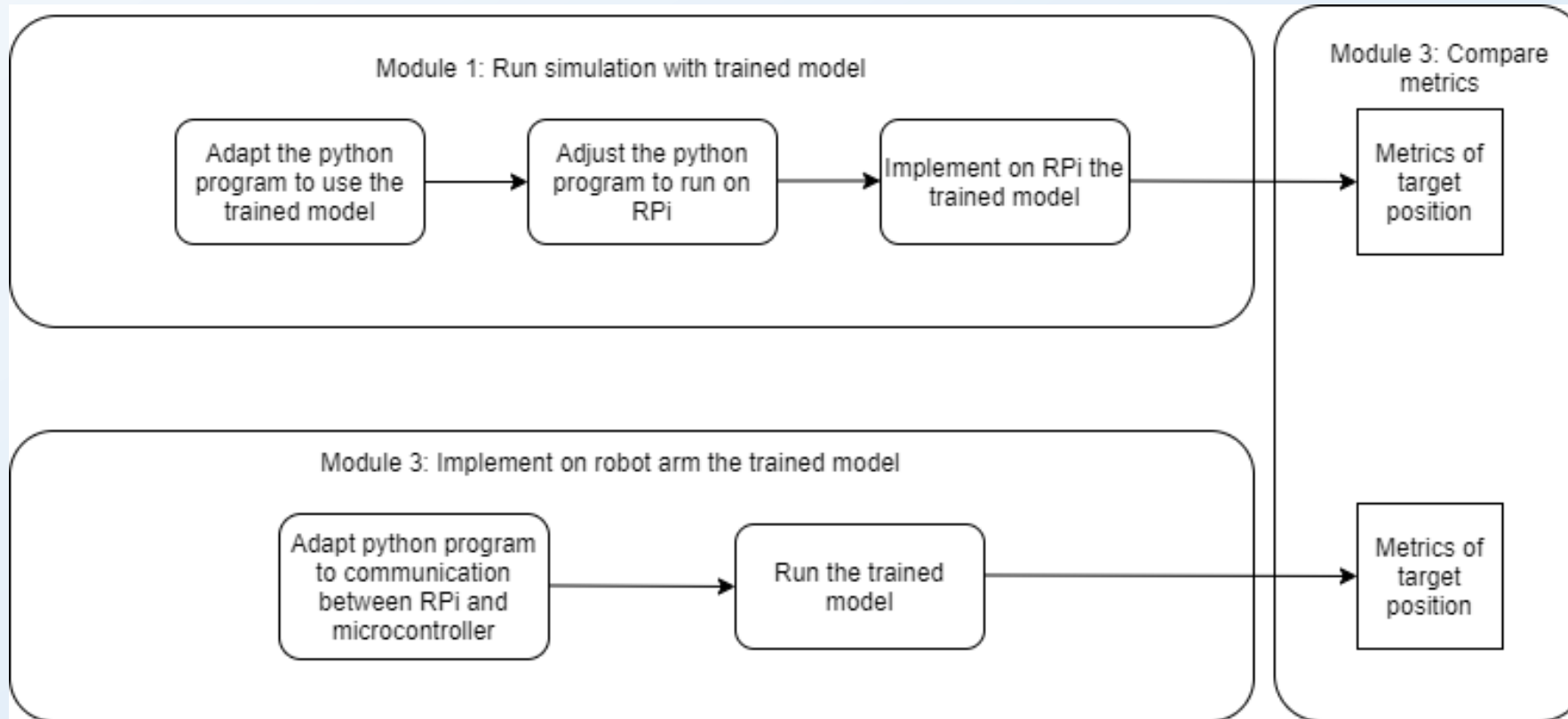
Theoretical Background

Theory 3 – Neural Network

- A neural network is a series of algorithms that endeavors to recognize underlying relationships in a set of data through a process that mimics the way the human brain operates.
- Artificial neural networks (ANNs) are comprised of a node layer, containing an input layer, one or more hidden layers, and an output layer.
- Each node, or artificial neuron, connects to another and has an associated weight and threshold.



Study Design



Study Design Overview: Methodology

Robot Agent

- Agent Randomly Explores State Action Spaces
- each axis can move $(-1,0,1)$
- 81 moves represent by 4 digits: 1012

Robot Arm

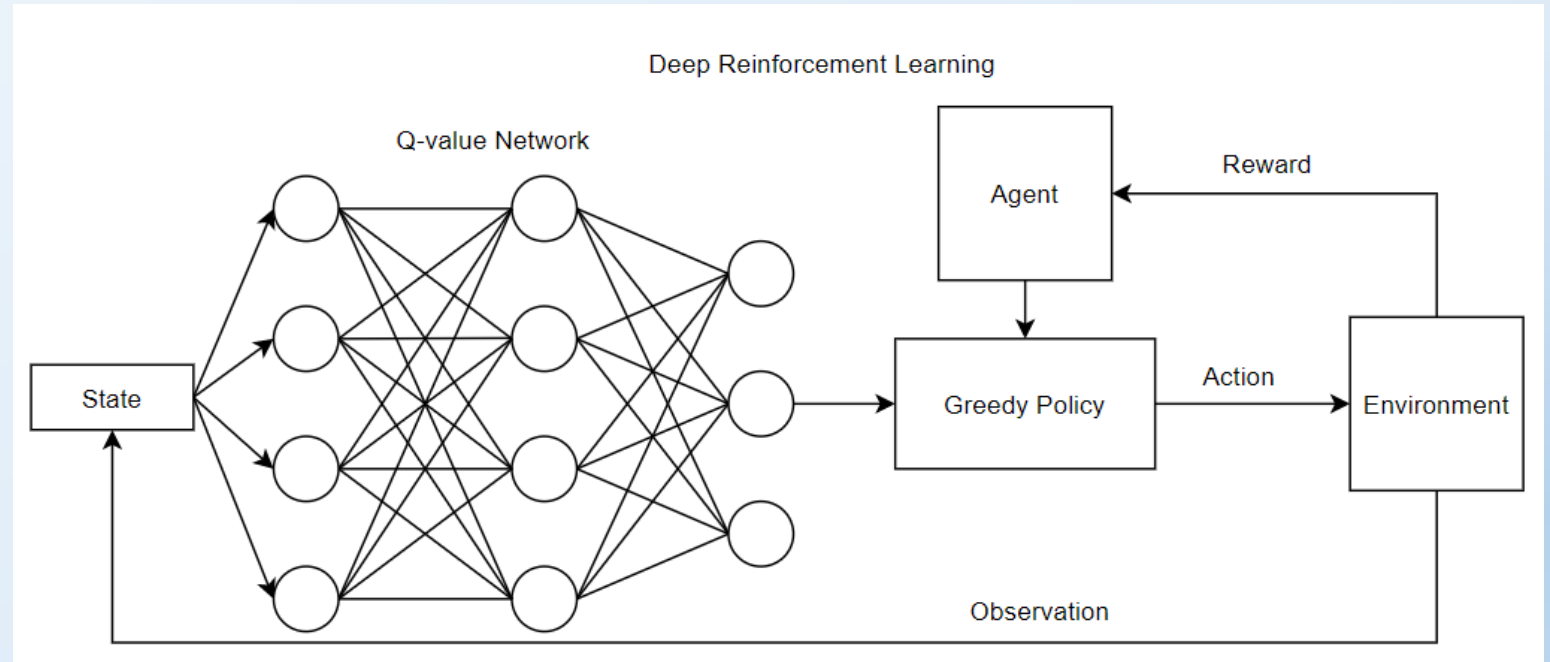
- 4 Degrees of Freedom Robot Arm
- R,R,R,R configuration
- Limits: 180, 85, 100, 180
- Controlled via Python interfacing with a microcontroller
- Working Space $R = (5 \text{ cm}, 26\text{cm})$, $\Phi = (-90,+90)$
- Servo Position Sensor used to Calculate Tip Position



Study Design Overview: Methodology

Neural Network

- TensorFlow 2.0
- Trained Model Deep Reinforcement Learning Network
- 7 inputs, 5 hidden layers, and 81 outputs
- Adam optimizer, Huber loss function



Study Design Overview: Methodology

Code and Processing

- Single Raspberry Pi
- 1 GB LPDDR2 SDRAM
- Broadcom BCM2837B0, Cortex-A53 (ARMv8) 64-bit Soc @ 1.4GHz



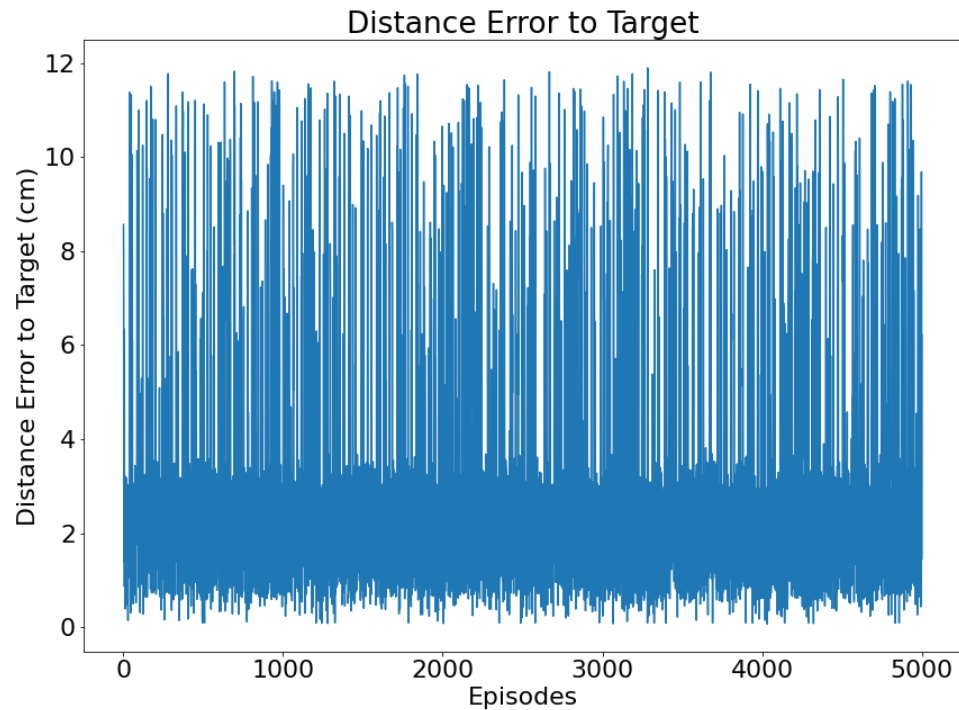
Results

Run simulation with trained model

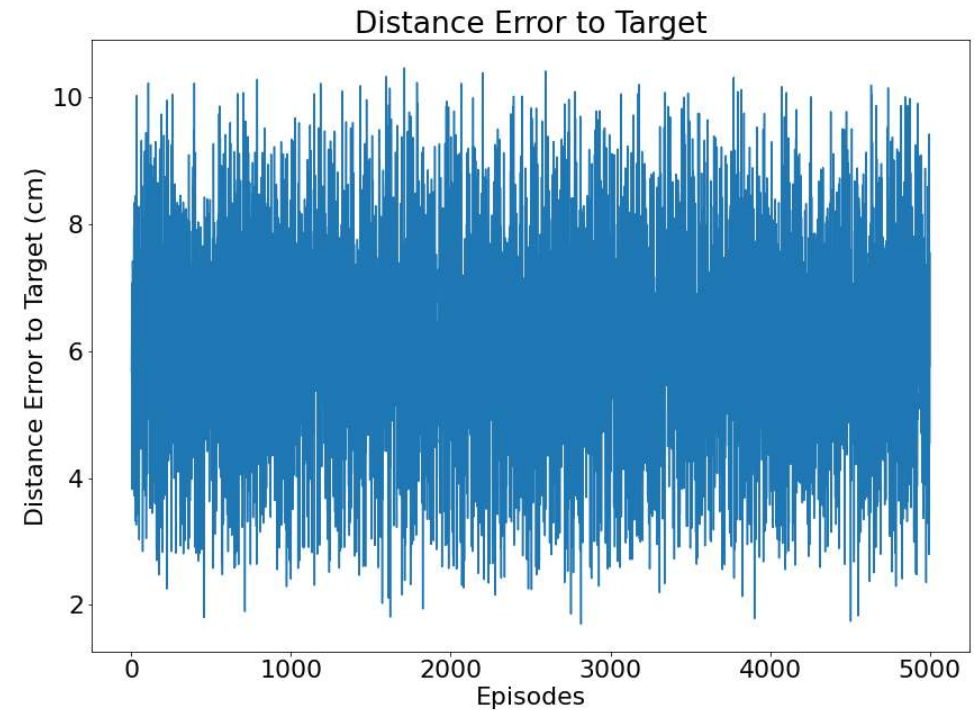
Statistic	ArbotiX	Dobot Magician
Average Distance Error	3.051 cm	6.155 cm
Average Euclidean Distance	3.520 cm	5.730 cm
Average Reward	11.797	1.653

Results

Run simulation with trained model



Simulation Distance error with ArbotiX Robot Arm

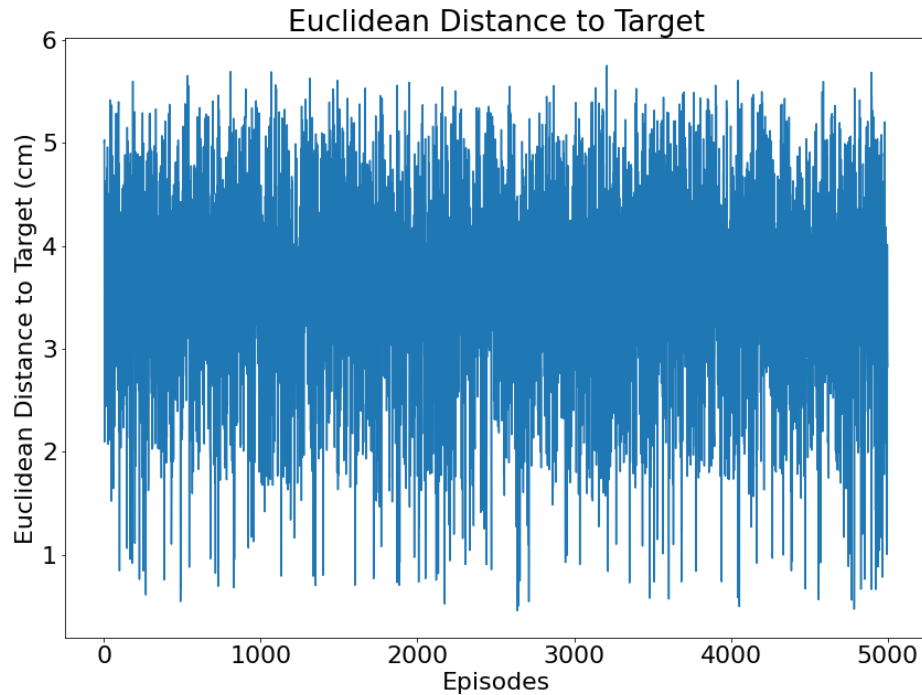


Simulation Distance error with Dobot Magician Robot Arm

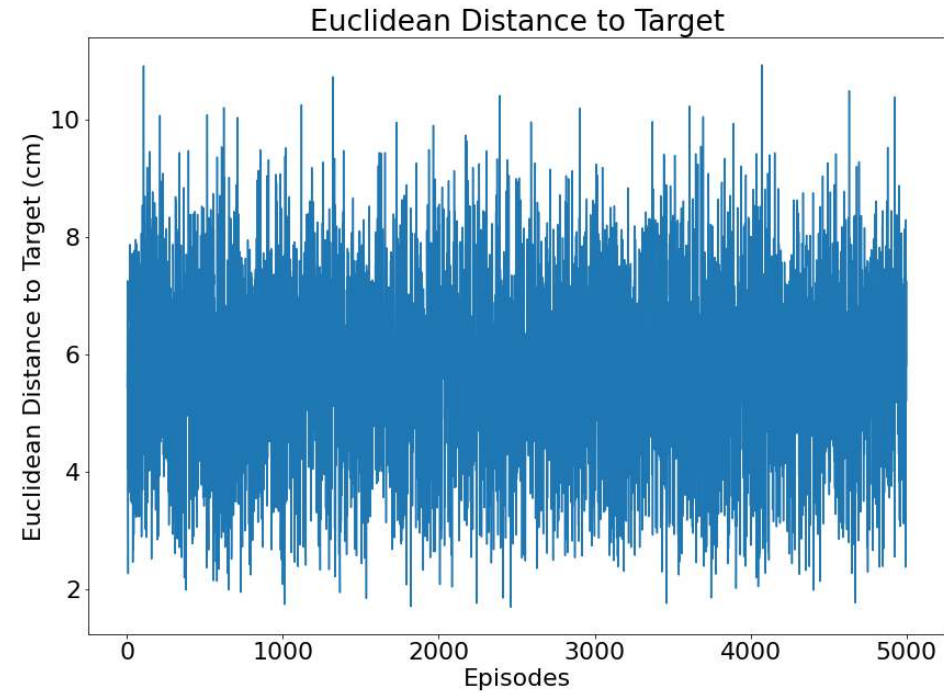
Distance Error Function: $error = r \times |\Delta\theta| + \sqrt{\Delta r^2 + \Delta h^2}$

Results

Run simulation with trained model



Euclidean Distance with ArbotiX Robot Arm



Euclidean Distance with Dobot Magician Robot Arm

Euclidean distance:
$$d(P_t, P_p) = \sqrt{(X_t - X_p)^2 + (Y_t - Y_p)^2 + (Z_t - Z_p)^2}$$

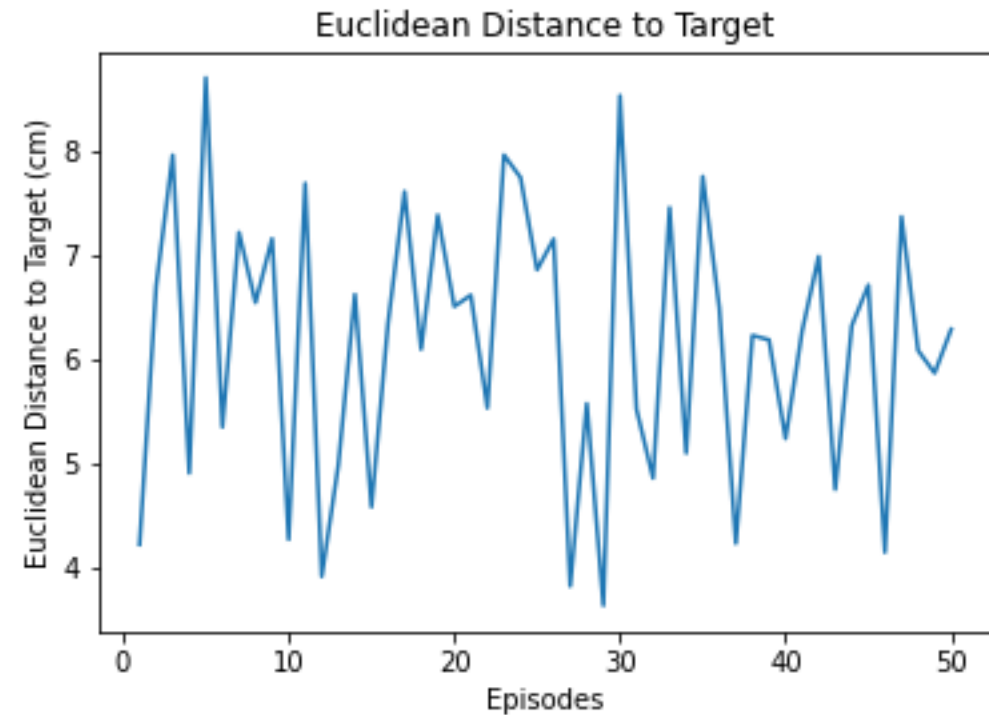
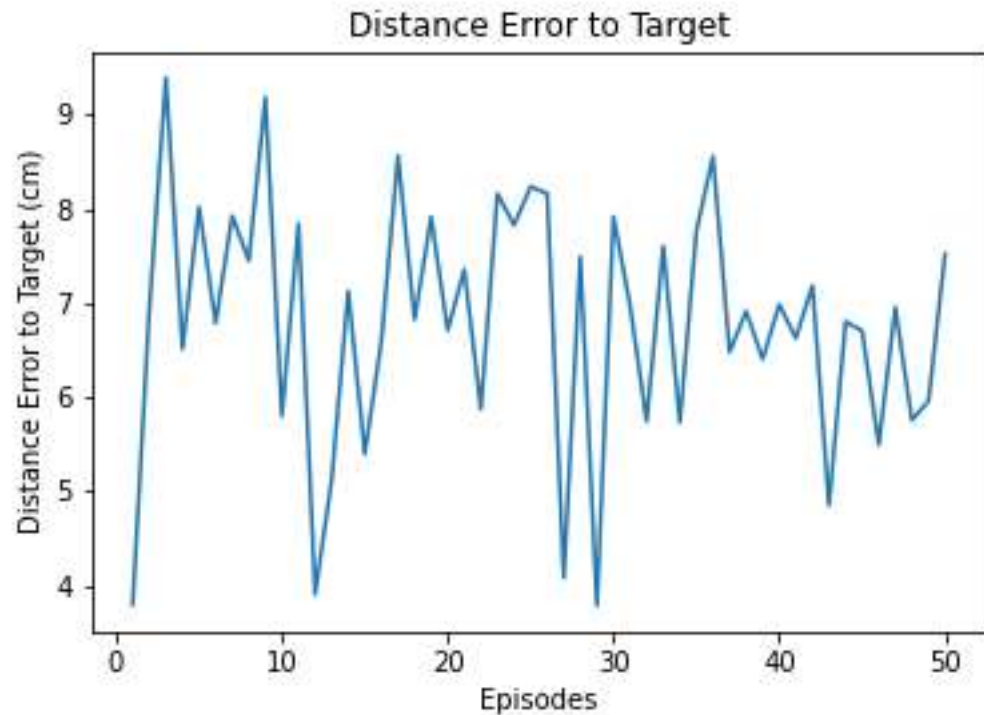
Results

Implemented on robot arm the trained model

- Implemented the model
 - Connect the Dobot Magician Robot Arm to RPi
 - Run the Python program
- Results:
 - Average Euclidean distance = 6.164 cm
 - Average Distance Error = 6.796 cm

Results

Implemented on robot arm the trained model



Conclusions

- We were able to implement simulation with the trained model.
- We were able to implement on hardware the trained model of 4 degrees of freedom robot arm.
- The trained model works well for the robot with which it was originally trained.
- When using a different 4 DOF Robot Arm the behavior was not as expected.

Future Work

- We can implement our trained model in different robot arm making the adjustment in the DH parameters to compare the accuracy to reach the target position.
- The accuracy will be better with Robot Arm that has similar characteristic to ArbotiX Robot Arm.
- Train the model with a bigger set of data we can have a good behavior and accuracy in the implementation.

Acknowledgements

Thanks to my project committee:

Dr. Sudhir Shrestha

Dr. Mohamed Salem

Dr. Nansong Wu

Special thanks to Dr. Shrestha for his advice

Thank you to Dr. Farahmand and the Department of Engineering Science

Thank you to Kate O'Brien for all her help with my paperwork as an international student.

Questions?